

K. RAMAKRISHNAN COLLEGE OF TECHNOLOGY

Tiruchirappalli, Tamil Nadu

SERVICENOW

PROJECT DOCUMENTATION

EMPLOYEE RAISE ISSUE

Record Producer & Service Portal Integration

Submitted by

HARINI N

Mail ID: harini.it23@krect.ac.in

ACADEMIC YEAR

2026–2027

S. No.	CONTENT	PAGE NO.
1	PROJECT INTRODUCTION	5
1.1	Project Overview	5
1.2	Problem Statement	5
1.3	Objectives	5
1.4	Technologies / ServiceNow Components Used	6
1.5	Project Architecture	7
2	ENVIRONMENT SETUP	8
2.1	Environment Used	8
3	PHASE 1 – CREATING A CUSTOM APPLICATION	9
3.1	Phase Overview	9
3.2	Implementation	9
3.3	Application Setup	9
4	PHASE 2 – CREATING THE CUSTOM TABLE AND FIELDS	11
4.1	Phase Overview	11
4.2	Implementation & Configuration	11
4.3	Creating Fields	12

S. No.	CONTENT	PAGE NO.
5	PHASE 3 – CREATING UI POLICIES AND DEPENDENCY	15
5.1	Phase Overview	15
5.2	UI Policy Configuration	15
5.3	Category–Subcategory Dependency	17
5.4	Dependent Behavior	18
6	PHASE 4 – CREATING A RECORD PRODUCER	20
6.1	Phase Overview	20
6.2	Record Producer Creation	20
6.3	Variable Configuration	21
6.4	Category–Subcategory Configuration	23
7	PHASE 5 – CREATING A NEW SERVICE PORTAL	25
7.1	Phase Overview	25
7.2	Portal Configuration	26
7.3	Employee Center Homepage	27
8	PHASE 6 – CREATING WIDGETS AND ADDING TO THE PAGE	29
8.1	Commercial Widget Creation	29

S. No.	CONTENT	PAGE NO.
8.2	Link Redirect Widget Creation	30
8.3	Adding Widgets to the Page	31
8.4	Final Employee Center Homepage	32
9	PHASE 7 – TESTING AND VALIDATION	34
9.1	Testing Approach	34
9.2	Test Scenarios	35
9.3	Category and Subcategory Validation	35
9.4	Record Creation Validation	35
9.5	End-to-End Validation	35
10	PHASE 8 – CONCLUSION	37
10.1	Key Outcomes	37
10.2	Final Result	37

1. PROJECT INTRODUCTION

PROJECT OVERVIEW

The Employee Raise Issue – Record Producer & Service Portal Integration project is a ServiceNow-based solution designed to provide employees with a simple and structured way to raise workplace-related issues. The solution uses a custom scoped application, custom table, UI Policies, field dependency, Record Producer, Service Portal, and custom widgets. Employees can access the Service Portal, open the Raise Employee Issue form, enter the required issue details, and submit the request. The submitted information is then captured as a record in the Employee Raise Issue custom table.

PROBLEM STATEMENT

In a large enterprise organization, employees may raise workplace-related issues through emails or informal communication channels. This can result in:

- Lack of standardized data collection.
- Delays in issue resolution.
- Limited visibility into issue status.
- Manual effort for HR and support teams.
- Inconsistent issue categorization and prioritization.
- Difficulty in maintaining and tracking employee requests.

Therefore, a centralized ServiceNow-based solution is required to provide a standardized, controlled, and easily accessible method for employees to raise and track issues.

OBJECTIVE

The main objective of this project is to develop a ServiceNow Employee Issue Management solution that enables employees to raise issues through a Service Portal using a Record Producer.

The specific objectives are to:

1. Create a custom scoped ServiceNow application.

2. Create an Employee Raise Issue custom table to store issue records.
3. Configure required fields such as Requester, Category, Subcategory, State, Priority, and Assignment Group.
4. Implement UI Policies for form validation and field restrictions.
5. Configure Category–Subcategory dependency.
6. Create a Record Producer for simplified issue submission.
7. Create and configure a Service Portal.
8. Develop custom Service Portal widgets.
9. Integrate the Record Producer with the Service Portal.
10. Test and validate the complete end-to-end workflow.

TECHNOLOGIES / SERVICENOW COMPONENTS USED

Component	Purpose
ServiceNow PDI	Development and testing environment
ServiceNow Studio	Creation of the scoped application
Custom Application	Contains the Employee Raise Issue solution
Custom Table	Stores submitted employee issues
UI Policies	Controls field behavior and validation
Field Dependency	Implements Category–Subcategory dependency
Record Producer	Provides the employee issue submission form
Service Portal	Provides the employee-facing interface
Service Portal Designer	Used to configure the portal layout
Service Portal Widgets	Provides customized portal functionality

HTML & CSS	Used to design and style custom widgets
Client Script	Handles widget navigation and actions
Reference Fields	Links users and assignment groups

PROJECT ARCHITECTURE

Employee Raise Issue – Record Producer & Service Portal Integration Architecture Diagram

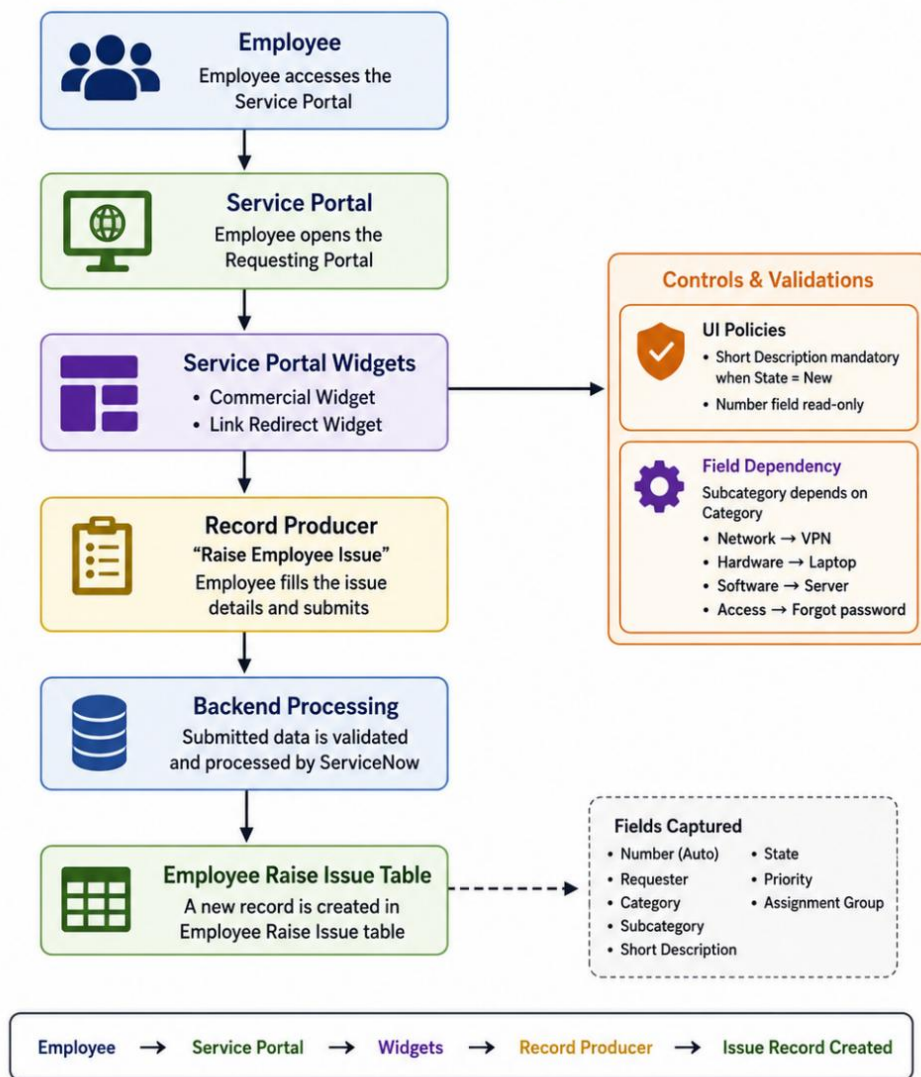


Figure 1.1: Architecture of the Employee Raise Issue ServiceNow Solution

2. ENVIRONMENT SETUP

Before starting the implementation, we set up and access the ServiceNow Personal Developer Instance (PDI). This instance is used as the development environment for creating and testing the complete Employee Raise Issue solution. We then work within the Employee Center application and its scoped application environment.

ENVIRONMENT USED

- **Platform:** ServiceNow
- **Development Environment:** Personal Developer Instance (PDI)
- **Application:** Employee Center
- **Application Type:** Scoped Application
- **Purpose:** Development, configuration, and testing of the Employee Raise Issue solution
- **Portal:** Employee Center
- **Development Tool:** ServiceNow Studio

The environment provided the required platform for developing, configuring, and testing the complete Employee Raise Issue solution.

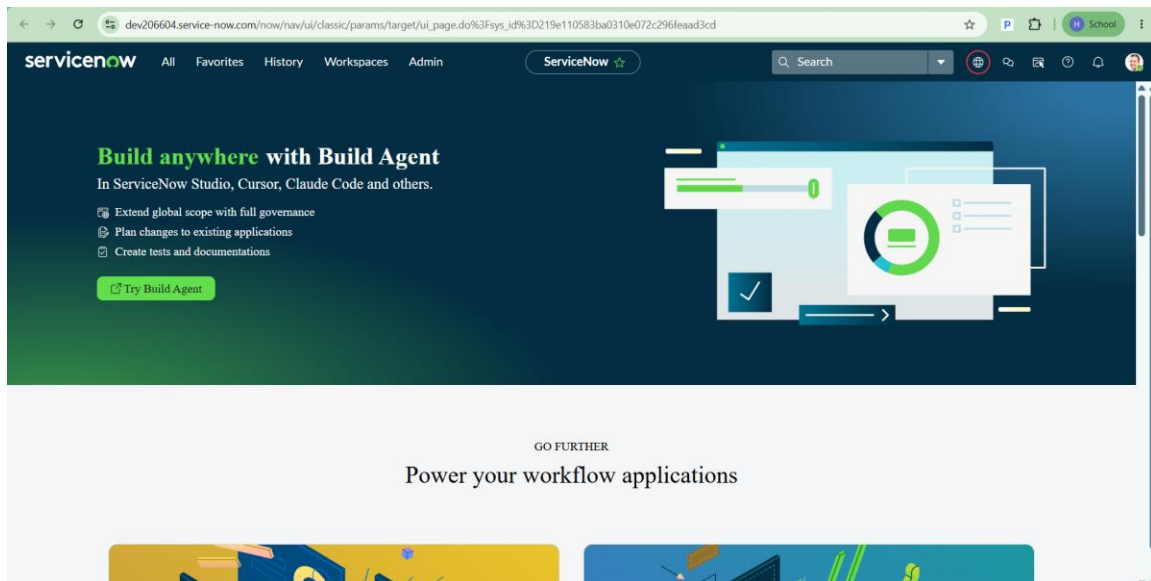


Figure 2.1: ServiceNow Personal Developer Instance used for the project

PHASE 1 – CREATING A CUSTOM APPLICATION

PHASE OVERVIEW

The first phase establishes the application foundation for the Employee Raise Issue solution in ServiceNow. A dedicated scoped application is created to organize and manage the project-specific tables, configurations, and functionality.

IMPLEMENTATION

A custom ServiceNow application named Employee Center was created with the following configuration:

Configuration	Value
Application Name	Employee Center
Application Type	Scoped
Purpose	Provides the application foundation for the Employee Raise Issue solution

The application scope provides a controlled environment in which the subsequent project components, including the custom table, fields, UI policies, Record Producer, and portal-related configurations, can be developed.

APPLICATION SETUP

The following activities were completed during the application setup:

- A dedicated Employee Center application was created in ServiceNow.
- The application was configured as a scoped application to maintain project-specific configurations within a controlled application boundary.
- The application provides the foundation for creating the Employee Raise Issue custom table and its related fields.
- The application scope is used to organize the UI Policies, Record Producer, and Service Portal components developed in the later phases.

- The application was created and managed through the ServiceNow Application Manager and ServiceNow Studio.
- The application environment was verified before proceeding with the creation of the custom table.

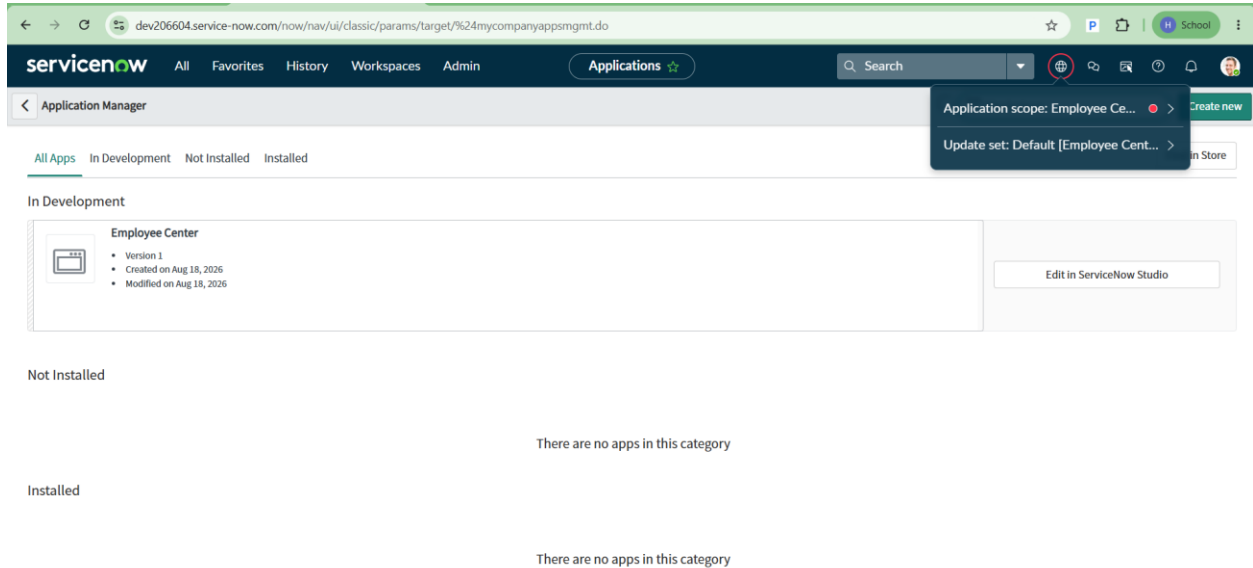


Figure 3.1: Employee Center scoped application created in ServiceNow Application Manager

RESULT

The Employee Center scoped application was successfully created and established as the foundation for the Employee Raise Issue project. The application environment is now ready for the creation of the custom data structure in Phase 2.

PHASE 2: CREATING THE CUSTOM TABLE AND FIELDS

PHASE OVERVIEW

The second phase establishes the custom data structure required to store employee-raised issues. A custom table was created within the Employee Center scoped application, along with the fields required to capture and manage issue information such as the requester, category, subcategory, short description, state, priority, and assignment group.

IMPLEMENTATION

A custom table named Employee raises issue was created within the Employee Center application. The table acts as the backend data structure for storing and managing employee issues submitted through the Record Producer.

CONFIGURATION

Configuration	Value
Application	Employee Center
Table Name	Employee Raises issue
Table Type	Custom Table
Purpose	Stores employee-raised issues and their associated details

TABLE AND FIELD SETUP

The following activities were completed during this phase:

- A custom Employee Raises Issue table was created within the Employee Center scoped application.
- The table was configured to store the information submitted by employees through the issue-raising process.
- Required fields were created to capture important issue details such as Requester, Category, Subcategory, Short Description, State, Priority, and Assignment Group.

- The Requester field was configured as a reference field to allow an employee/user to be selected.
- The Assignment Group field was configured to support the assignment of issues to the appropriate group.
- The State and Priority fields were included to support issue tracking and management.
- The Category and Subcategory fields were created to organize issues based on their type.
- The custom table provides the backend structure required for the Record Producer and Service Portal functionality implemented in later phases.

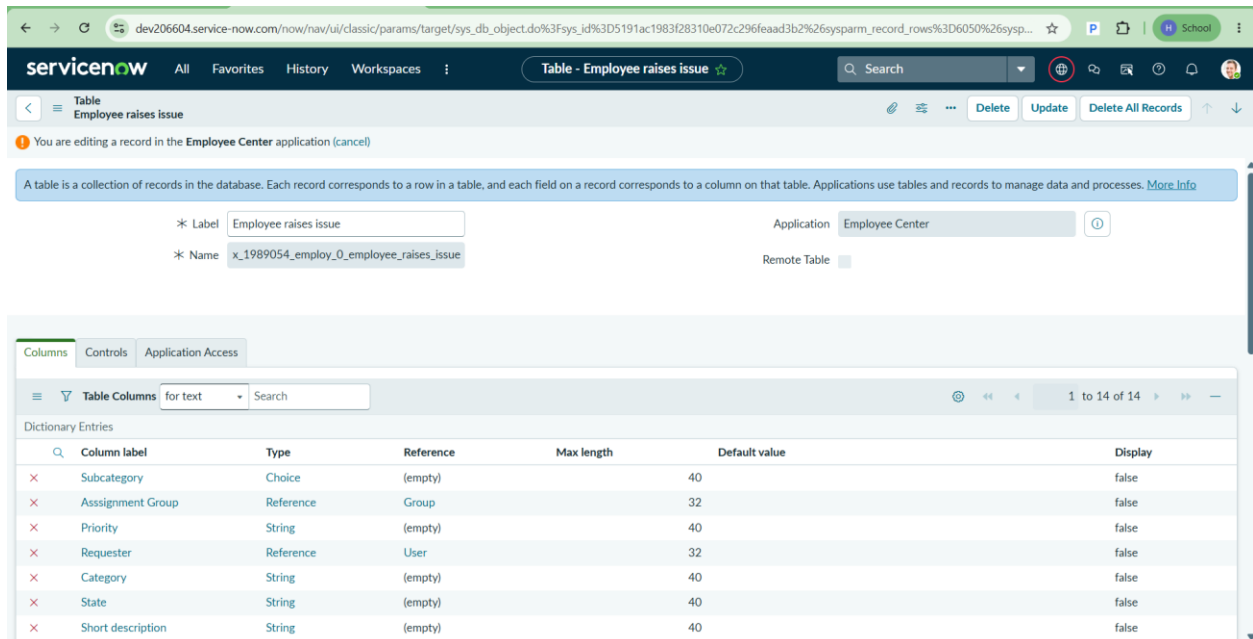


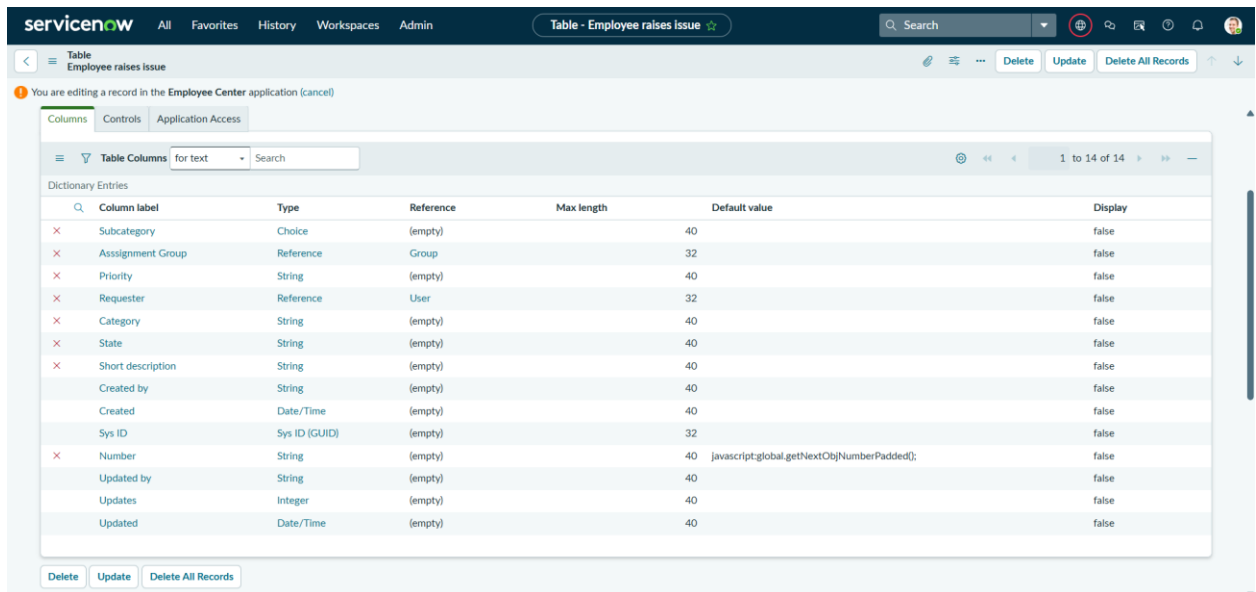
Figure 4.1: Employee Raise issue custom table and configured fields

FIELDS CREATED

The custom table contains fields required to capture the information submitted by an employee.

Field	Type	Purpose
Number	Auto-number	Automatically identifies each issue

Requester	Reference (sys_user)	Identifies the employee who raised the issue
Category	Choice	Defines the issue category
Subcategory	Choice	Defines the specific issue type
Short description	String	Provides a brief description of the issue
State	Choice	Tracks the issue status
Priority	Choice	Indicates the urgency of the issue
Assignment Group	Reference (sys_user_group)	Identifies the group responsible for the issue



The screenshot shows the ServiceNow interface for configuring the 'Employee raises issue' table. The 'Table Columns' tab is selected, showing a list of fields. The fields are as follows:

Column label	Type	Reference	Max length	Default value	Display
Subcategory	Choice	(empty)	40		false
Assignment Group	Reference	Group	32		false
Priority	String	(empty)	40		false
Requester	Reference	User	32		false
Category	String	(empty)	40		false
State	String	(empty)	40		false
Short description	String	(empty)	40		false
Created by	String	(empty)	40		false
Created	Date/Time	(empty)	40		false
Sys ID	Sys ID (GUID)	(empty)	32		false
Number	String	(empty)	40	javascript:global.getNextObj(NumberPadded);	false
Updated by	String	(empty)	40		false
Updates	Integer	(empty)	40		false
Updated	Date/Time	(empty)	40		false

Figure 4.2: Fields configured for the Employee Raise issue table

1. The Requester field uses a reference to the sys_user table so that an existing ServiceNow user can be selected.
2. The Category field contains predefined choices such as:
 - Network

- Software
 - Hardware
 - Access
3. The Subcategory field is configured as a dependent choice field so that its available options are filtered according to the selected Category.

The screenshot displays the ServiceNow interface for configuring a Dictionary Entry - Subcategory. The top navigation bar includes 'servicenow', 'All', 'Favorites', 'History', 'Workspaces', and 'Admin'. The breadcrumb trail shows 'Dictionary Entry - Subcategory'. A search bar is present on the right. Below the navigation, a message states: 'This record is in the Employee Center application, but Employee Center is the current application. To edit this record click here.' A blue informational box explains that a dictionary entry manages how ServiceNow stores data in tables and fields, and provides instructions on selecting a Table and field Type, entering a column label, and setting a Max length for text String type fields. The configuration form includes the following fields:

- Table:** Employee raises issue [x_1989054_employee_0_employee_raises_issue]
- Type:** Choice
- Column label:** Subcategory
- Column name:** u_choice_4
- Application:** Employee Center
- Active:** ☒
- Function field:** ☐
- Mandatory:** ☐
- Display:** ☐
- Read only option:** -- None --

Below these fields, a blue box states: 'Alters the behavior of a field or functionality that depends on the field. [More Info](#)'. The **Attributes** section shows 'edge_encryption_enabled=true'. At the bottom, there are four tabs: 'Choice List Specification', 'Dependent Field' (active), 'Calculated Value', and 'Default Value'. The 'Dependent Field' tab contains a blue box stating: 'Specify a field whose value is used to determine choice list options for this field. [More Info](#)'. Below this, the 'Use dependent field' checkbox is checked, and the 'Dependent on field' is set to 'u_choice_3'.

Figure 4.3: Category and Subcategory field configuration

RESULT

The Employee raises issue custom table was successfully created within the Employee Center scoped application. The required fields were configured to establish the backend data structure for the Employee Raise Issue solution. This table is subsequently used by the Record Producer to store employee-submitted issue records.

PHASE 3: CREATING UI POLICIES AND DEPENDENCY

PHASE OVERVIEW

The third phase focuses on configuring UI Policies, controlling the behavior of the Number field, and establishing a dependency between the Category and Subcategory fields in the Employee Raise issue form. These configurations ensure that the form behaves consistently and that employees are presented with relevant field options while submitting an issue.

IMPLEMENTATION

- The required UI Policy configurations were implemented for the Employee Raise issue table.
- The UI Policy was configured with the condition State is New. The Short description field was configured as Mandatory through the associated UI Policy Action.
- The Number field was configured as Read only so that users cannot modify the automatically generated issue number.
- In addition, the Subcategory field was configured through its Dictionary settings to depend on the Category field. This establishes Category as the controlling field for Subcategory.
- These configurations implement the required UI behavior and field dependency specified for the Employee Raise issue form.

UI POLICY CONFIGURATION

Configuration	Value
Application	Employee Center
Table	Employee Raise issue
UI Policy Condition	State is New
Field Action 1	Short description – Mandatory
Field Action 2	Number – Read only

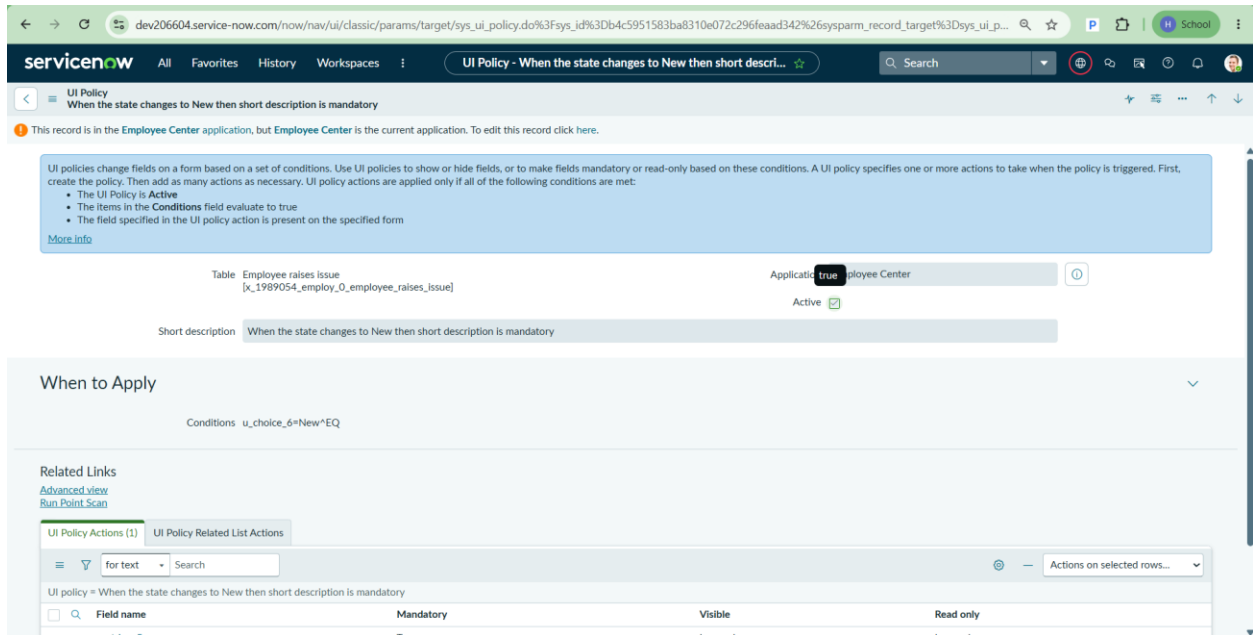


Figure 5.1: Employee Raise Issue UI Policy configuration

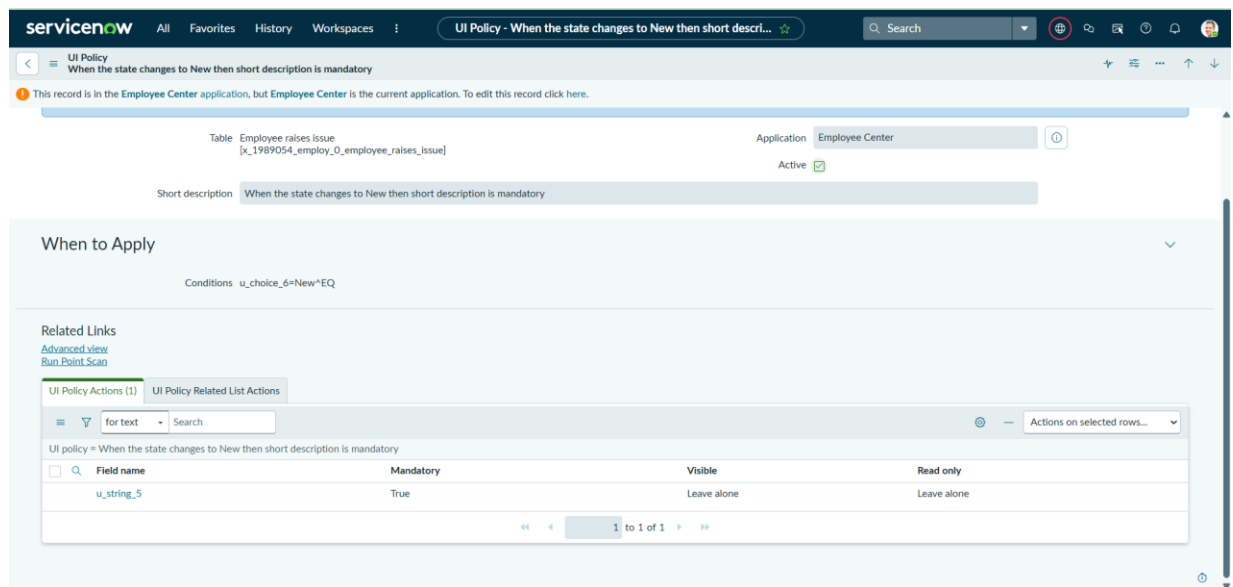


Figure 5.2: Configuring Short Description UI Policy Action

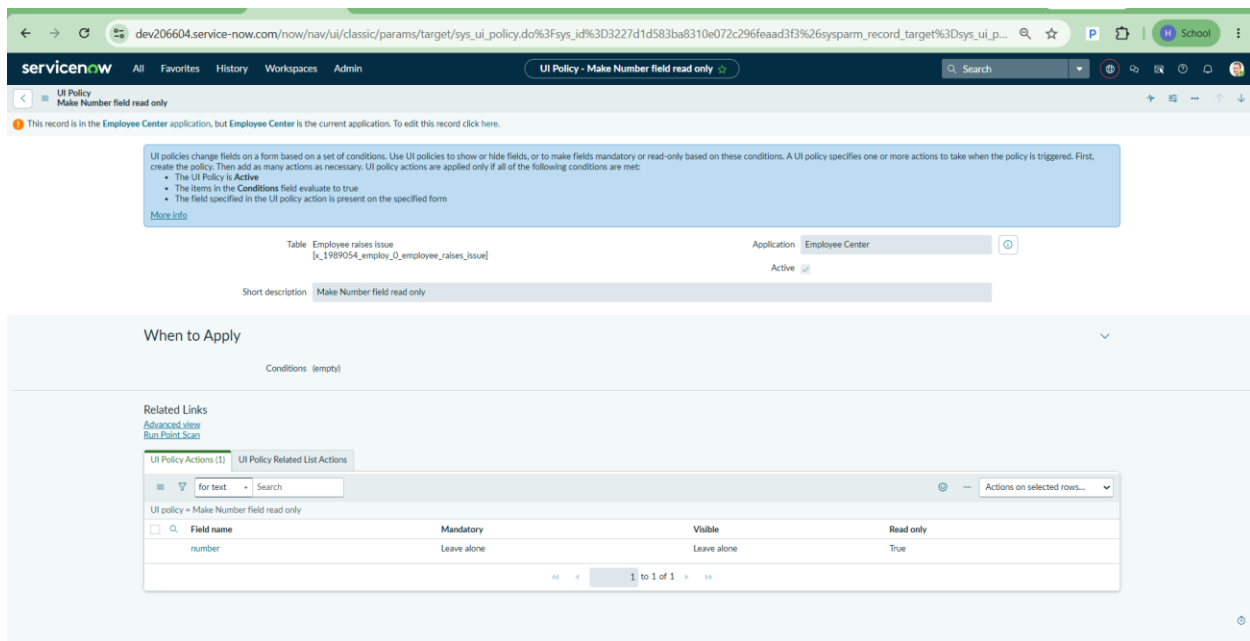


Figure 5.3: Configuring Number Field as Read Only

UI POLICY BEHAVIOR

When the defined UI Policy condition is satisfied, the Short description field is made mandatory. This ensures that employees provide a description when submitting a new issue. The Number field is configured as Read only, preventing users from modifying the automatically managed issue number.

CATEGORY-SUBCATEGORY DEPENDENCY

The **Subcategory** field was configured through its Dictionary settings to depend on the **Category** field. In the advanced configuration, **Category** was specified as the dependent field for Subcategory. This configuration ensures that the available Subcategory choices are filtered according to the Category selected by the employee.

Category	Subcategory
Network	VPN
Hardware	Laptop
Software	Server

Access	Forgot Password
--------	-----------------

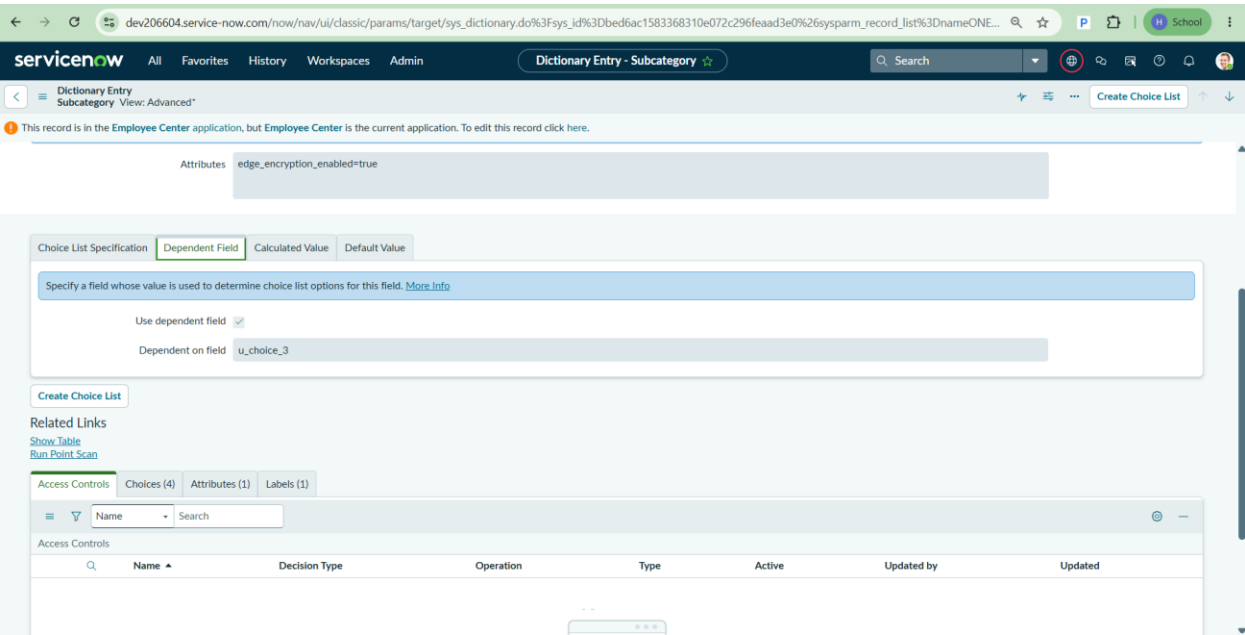


Figure 5.4: Configuring Subcategory Dependency

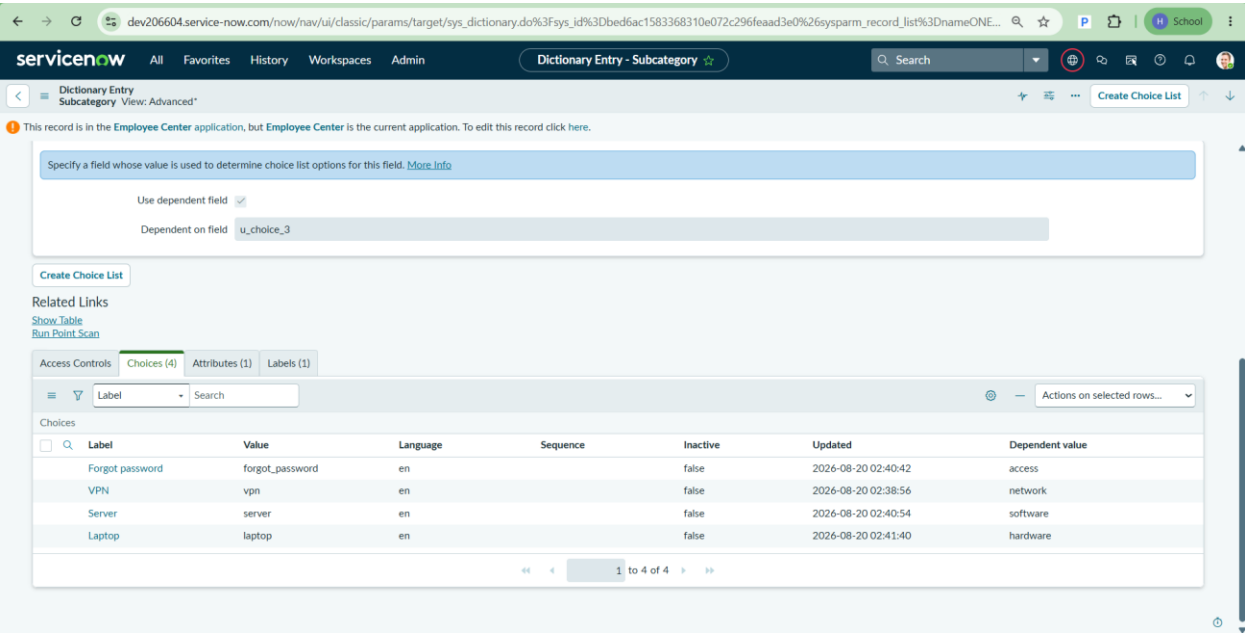


Figure 5.5: Configured Dependent Choices

EXAMPLE OF DEPENDENT BEHAVIOR

The Subcategory field changes based on the selected Category.

- Network → VPN
- Hardware → Laptop
- Software → Server
- Access → Forgot Password

For example, when Network is selected as the Category, VPN is displayed as the corresponding Subcategory. Similarly, selecting Hardware displays Laptop, selecting Software displays Server, and selecting Access displays Forgot Password. This dependency prevents unrelated Subcategory options from being displayed to the employee.

RESULT

The UI Policy and field dependency configurations were successfully implemented for the Employee Raise issue application.

The UI Policy controls the required and read-only behavior of the configured fields, while the Category field controls the available Subcategory choices. As a result, the issue submission form provides a more controlled and consistent experience for employees.

PHASE 4: CREATING A RECORD PRODUCER

PHASE OVERVIEW

The fourth phase focuses on creating a Record Producer that provides employees with a structured form for raising issues. The Record Producer collects the required information and creates a corresponding record in the Employee Raise issue custom table.

IMPLEMENTATION

A Record Producer named Raise Employee Issue was created within the Employee Center application. The Record Producer was configured to create records in the Employee Raise issue table. The required variables were added to the Record Producer to collect issue-related information from the employee.

CONFIGURATION

Configuration	Value
Application	Employee Center
Record Producer Name	Raise Employee Issue
Target Table	Employee Raise issue
Purpose	Provides an employee-facing form for submitting issues

RECORD PRODUCER SETUP

The following activities were completed during this phase:

- A Raise Employee Issue Record Producer was created.
- The Record Producer was configured to create records in the Employee Raise issue table.
- Required variables were added to collect employee and issue details.
- Variables such as Requester, Category, Subcategory, Short Description, and Description were configured.

- The Category and Subcategory fields use the dependency configured in the previous phase.
- The submitted information is mapped to the corresponding fields in the Employee Raise issue table.
- The Record Producer provides a simple interface for employees to submit their issues without directly accessing the backend table.

The screenshot shows the ServiceNow interface for configuring a Record Producer. The browser address bar shows a URL from dev206604.service-now.com. The page title is 'Record Producer - Raise Employee'. A notification states: 'This record is in the Employee Center application, but Employee Center is the current application. To edit this record click here.' Below this, there's a 'Manage Attachments' section with one attachment 'images.jpg [rename][download]'. A blue banner reads: 'Build and modify items faster with the improved Catalog Builder.' The configuration fields include: Name (Raise Employee), Application (Employee Center), Table name (Employee raises issue [x_1989054_employee_0_employee_raises_issue]), Active (checked), State (None), Checked out (None), and Owner (System Administrator). Below these are tabs for 'What it will contain', 'Accessibility', 'Generated Record Data', and 'Portal Settings'. The 'What it will contain' tab is active, showing a section to 'Add a short description and a full description'. The 'Short description' is 'Employee issue reporting form' and the 'Description' is 'Use this form to report workplace issues such as network, hardware, software, or access problems.' An illustration of a wrench and a screwdriver is shown below the description.

Figure 6.1: Raise Employee Issue Record Producer Configuration

VARIABLES CONFIGURED

The following variables were configured in the Record Producer:

Variable	Type	Purpose
Requester	Reference	Identifies the employee raising the issue
Category	Choice	Allows selection of the issue category
Subcategory	Choice	Allows selection of the relevant issue type
Description	Text	Captures details about the issue

- The **Requester** variable is associated with the ServiceNow user information, allowing the employee/requester to be identified.
- The **Category** variable contains the predefined issue categories:
 - Network
 - Software
 - Hardware
 - Access
- The **Subcategory** variable uses the dependency configured in Phase 3 so that the available choices correspond to the selected Category.

Question	Field	Type	Order
Who is raising this issue?	u_reference_2	Reference	100
What type of issue are you facing?	u_choice_3	Select Box	200
What is the specific issue?	u_choice_4	Select Box	300
What is the current status of the issue?	u_choice_6	Select Box	400
Briefly describe your issue	u_string_5	Multi Line Text	500

Figure 6.2: Variables Configured in the Raise Employee Issue Record Producer

RECORD PRODUCER FORM

The Record Producer provides a simple form through which employees can enter their issue information. After the employee submits the form, ServiceNow creates a corresponding record in the **Employee Raise issue** table.

Figure 6.3: Employee-Facing Raise Employee Issue Form

CATEGORY AND SUBCATEGORY DEPENDENCY

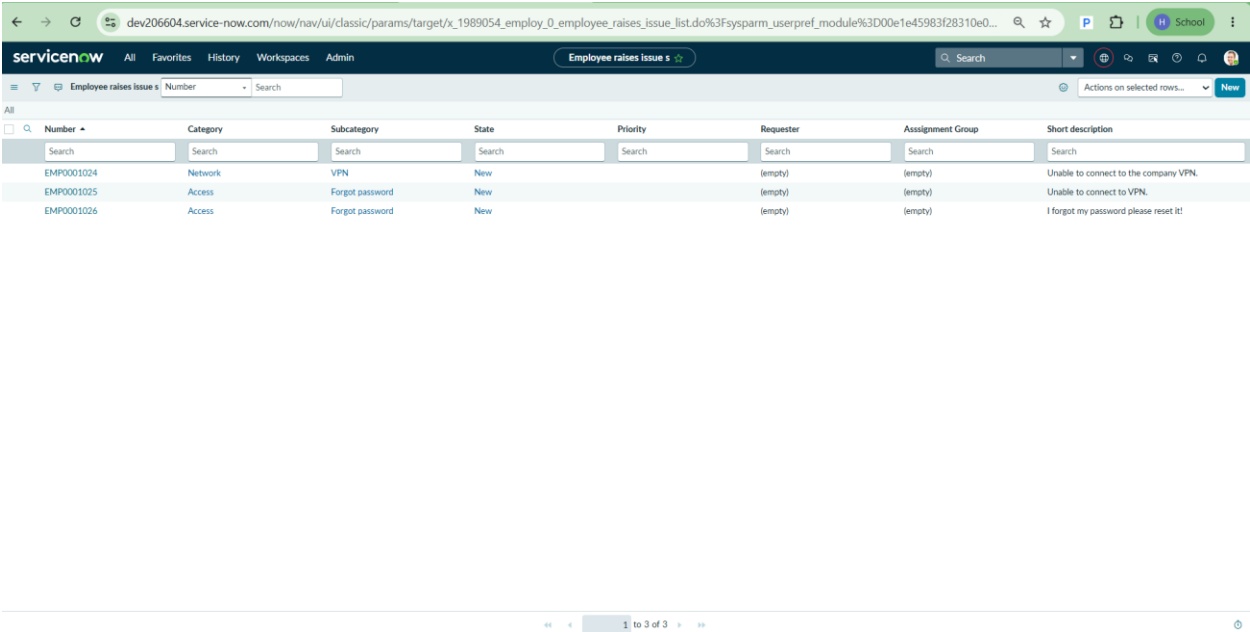
The Subcategory depends on the selected Category. Only the relevant subcategory is shown.

- Network → VPN
- Hardware → Laptop
- Software → Server
- Access → Forgot Password

Figure 6.4: Record Producer configuration with dependent fields

SUBMISSION AND RECORD CREATION

When the employee submits the Record Producer form, ServiceNow processes the submitted variable values and creates a new record in the configured target table, Employee Raise issue. The Record Producer therefore acts as the interface between the employee-facing form and the backend custom table. The submitted issue information is stored in the corresponding fields of the newly created record.



The screenshot shows the ServiceNow interface for the 'Employee raises issue s' table. The table has columns for Number, Category, Subcategory, State, Priority, Requester, Assignment Group, and Short description. Three records are displayed:

Number	Category	Subcategory	State	Priority	Requester	Assignment Group	Short description
EMP0001024	Network	VPN	New		(empty)	(empty)	Unable to connect to the company VPN.
EMP0001025	Access	Forgot password	New		(empty)	(empty)	Unable to connect to VPN.
EMP0001026	Access	Forgot password	New		(empty)	(empty)	I forgot my password please reset it!

Figure 6.5: Submitted Issue Record in Employee Raise Issue Table

RESULT

The **Raise Employee Issue** Record Producer was successfully created and configured. It provides the required interface for employees to submit issues and transfers the submitted information to the custom Employee Raise issue table.

PHASE 5: CREATING A NEW SERVICE PORTAL

PHASE OVERVIEW

The fifth phase focuses on creating and configuring a dedicated **Service Portal** for the Employee Raise Issue solution. The portal provides the employee-facing interface through which users can access the issue submission functionality.

IMPLEMENTATION

A dedicated Employee Center Service Portal was configured within the ServiceNow environment to provide an employee-facing interface for the Employee Raise Issue application. The portal acts as the presentation layer between employees and the backend ServiceNow application components created in the previous phases. It provides a centralized interface through which employees can access the issue-raising functionality without directly interacting with the backend application or database.

CONFIGURATION

Configuration	Value
Application	Employee Center
Portal	Employee Center
Purpose	Provides an employee-facing interface for accessing issue-related functionality
Page	Employee Center Homepage

PORTAL SETUP

The following activities were completed during the portal configuration:

- The Employee Center portal was configured as the employee-facing interface.
- The Employee Center Homepage was used as the main landing page.
- The portal was prepared to display the required issue-related widgets.

- Navigation was configured to allow employees to access the Raise an Issue functionality.
- The portal provides a simple and user-friendly interface for employees.
- The configured portal connects the frontend interface with the issue submission functionality developed in the previous phases.

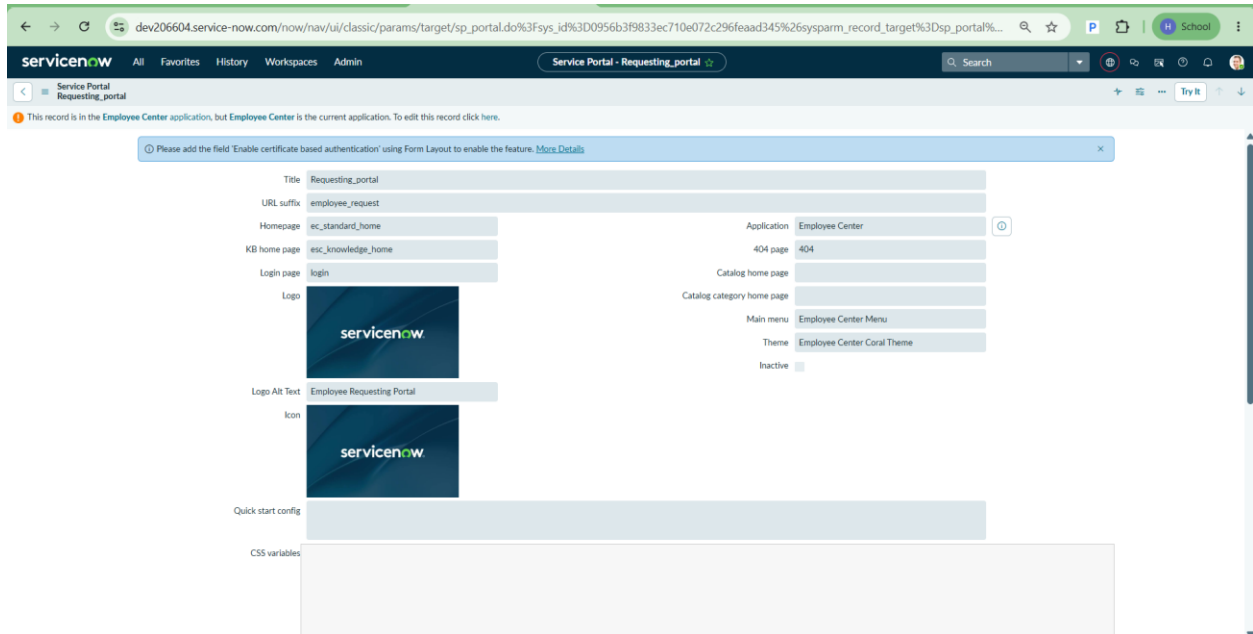


Figure 7.1: Employee Center Service Portal Configuration

The Employee Center portal was configured with the required portal settings and homepage structure. The configuration establishes the basic environment required for providing employee-facing access to the Employee Raise Issue functionality.

PORTAL CONFIGURATION

The Employee Center portal was opened and configured using the ServiceNow portal/page configuration options. The portal provides a centralized interface where employees can access the **Raise Employee Issue** functionality without directly interacting with the backend application or database.

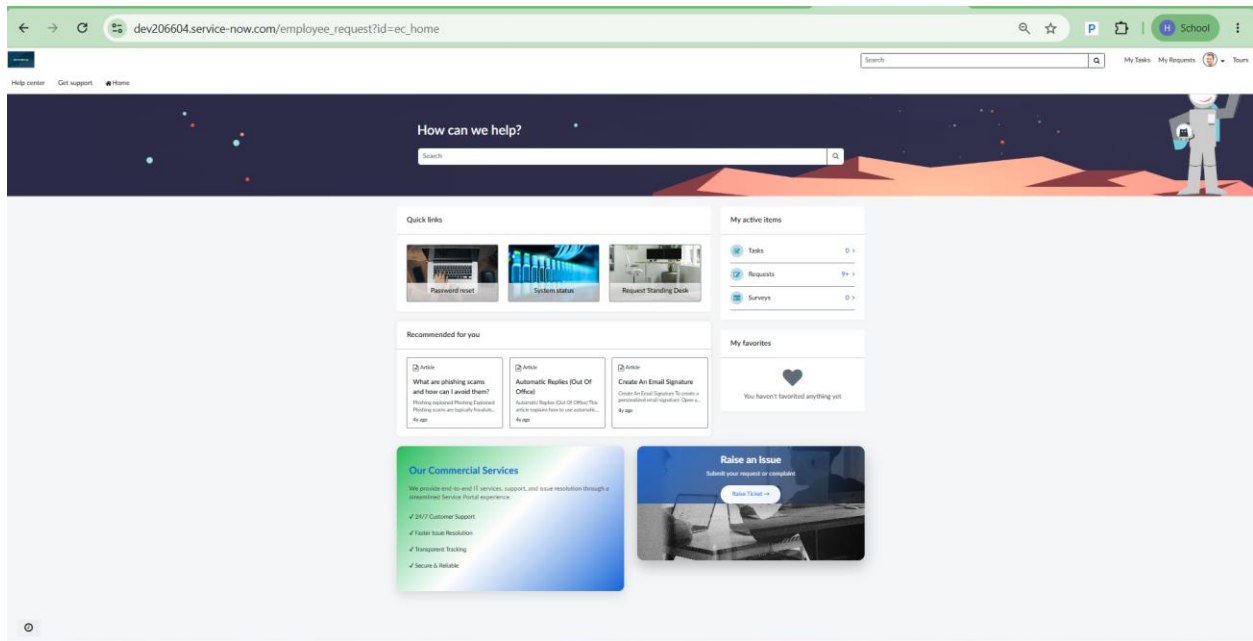


Figure 7.2: Employee Center Portal Homepage

RAISE AN ISSUE INTEGRATION

The **Raise an Issue** option was integrated into the Employee Center homepage so that employees can easily access the issue-raising functionality from the portal.

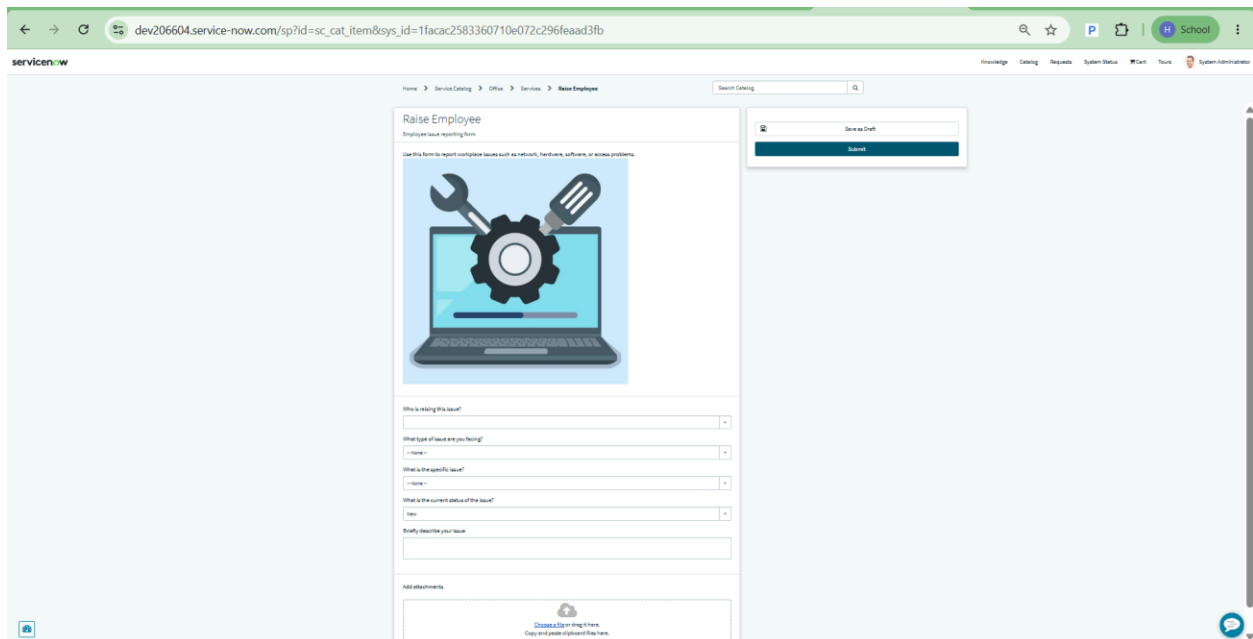


Figure 7.3: Raise an Issue option in Employee Center

The **Raise an Issue** section provides a clear entry point for employees to submit requests or complaints. Selecting the **Raise Ticket** option directs the employee to the issue submission functionality configured in the earlier phases.

RESULT

The Employee Center Service Portal was successfully configured as the employee-facing interface for the Employee Raise Issue solution. The portal homepage was prepared with the required Raise an Issue access point, enabling employees to easily navigate to the issue submission functionality. The portal provides a user-friendly presentation layer while the backend application components remain managed within the ServiceNow platform.

PHASE 6: CREATING WIDGETS AND ADDING TO THE PAGE

PHASE OVERVIEW

The sixth phase focuses on creating the required ServiceNow widgets and adding them to the existing Employee Center Homepage. The widgets provide the employee-facing interface for accessing the Employee Raise Issue functionality.

Two custom widgets were created:

- Commercial widget
- Link redirect widget

The widgets were then added to the Employee Center Homepage using a 6 × 6 layout.

IMPLEMENTATION

- Two custom Service Portal widgets were created using the Service Portal Widget Editor.
- The Commercial widget was created as the first widget and configured using HTML and CSS to display the commercial services information.
- The Link redirect widget was created as the second widget. It was configured using HTML, CSS, and Client Script to provide navigation to the Raise Employee Issue Record Producer.
- After creating both widgets, they were added to the existing Employee Center Homepage using the Service Portal Designer.

1. COMMERCIAL WIDGET CREATION

The Commercial widget was created using the Service Portal Widget Editor. HTML and CSS were added to define its content, appearance, and visual effects.

The widget displays information related to:

- 24/7 Customer Support
- Faster Issue Resolution
- Transparent Tracking
- Secure & Reliable Services

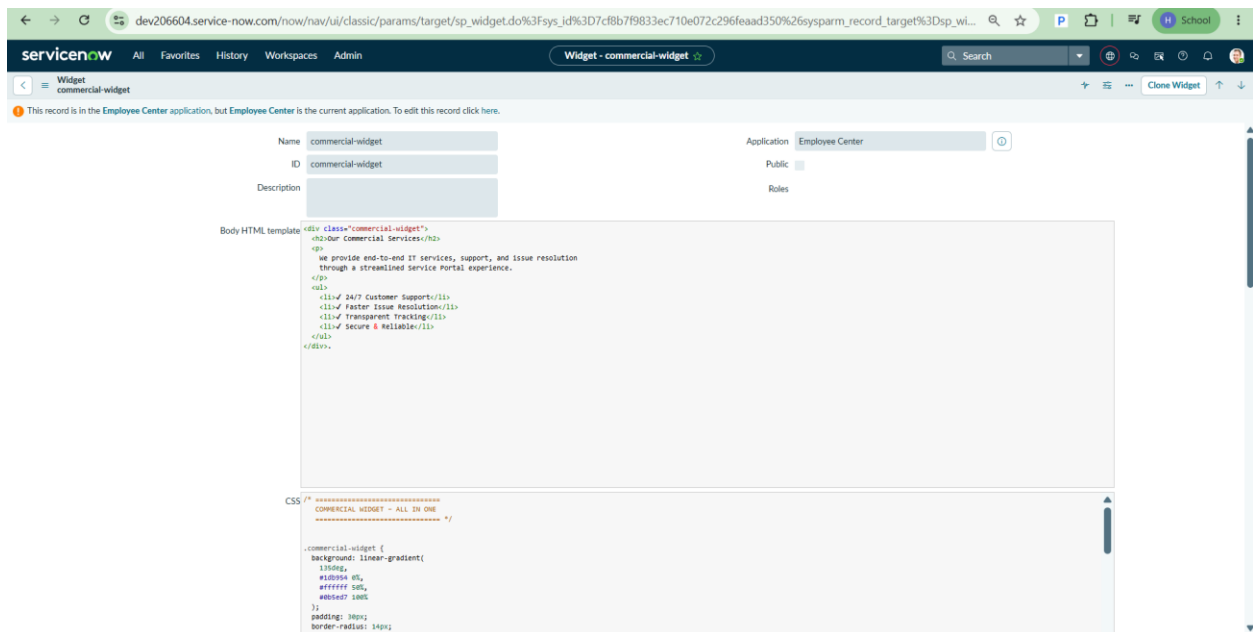


Figure 8.1: Commercial widget configuration

2. LINK REDIRECT WIDGET CREATION

The Link redirect widget was created to provide direct navigation to the Employee Raise Issue form.

The widget was configured using HTML, CSS, and Client Script. The Client Script uses the Record Producer sys_id to redirect the employee to the Raise Employee Issue submission form.

The navigation flow is:

Employee Center → Raise an Issue → Employee Raise Issue Form

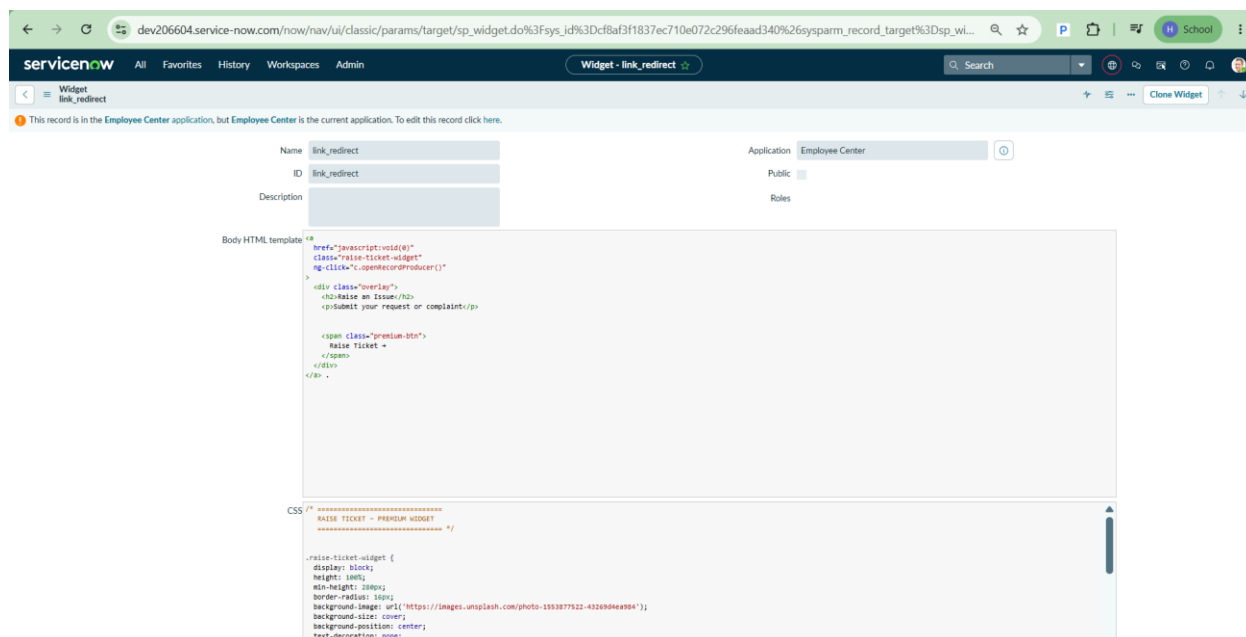


Figure 8.2: Link redirect widget configuration

3. ADDING WIDGETS TO THE PAGE

- The existing Employee Center Homepage (ec_home) was opened using the Service Portal Designer.
- A 6×6 container was created to organize the two widgets. The container was divided into two equal columns.

CONFIGURATION

Configuration	Value
Portal	Employee Center
Page	Employee Center Homepage (ec_home)
Container	Employee Center Homepage – Container 3
Row	Row 1

Layout	6 × 6
Column 1	Size 6 – commercial-widget
Column 2	Size 6 – link_redirect
Purpose	Provides employees with access to the Employee Raise Issue functionality

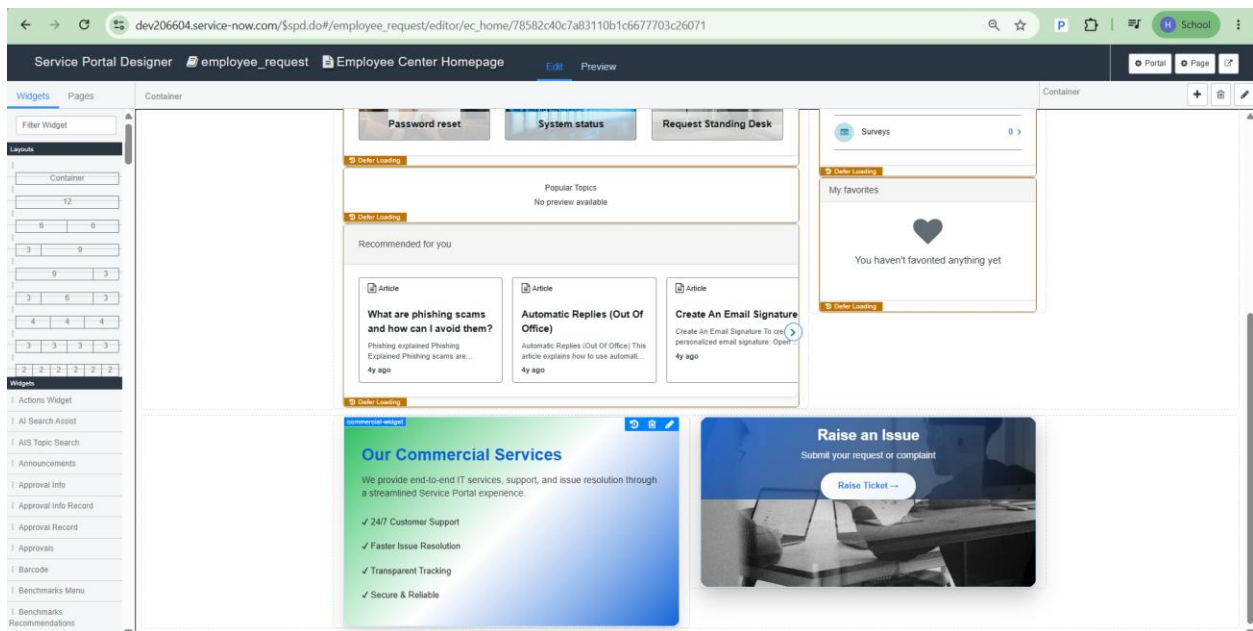


Figure 8.3: Adding the two widgets to the Employee Center Homepage

4. FINAL PAGE

- After adding the widgets, the Employee Center Homepage was saved and opened through the employee-facing portal to verify the final appearance.
- The completed page displays the two configured widgets and provides employees with direct access to the issue-raising functionality.

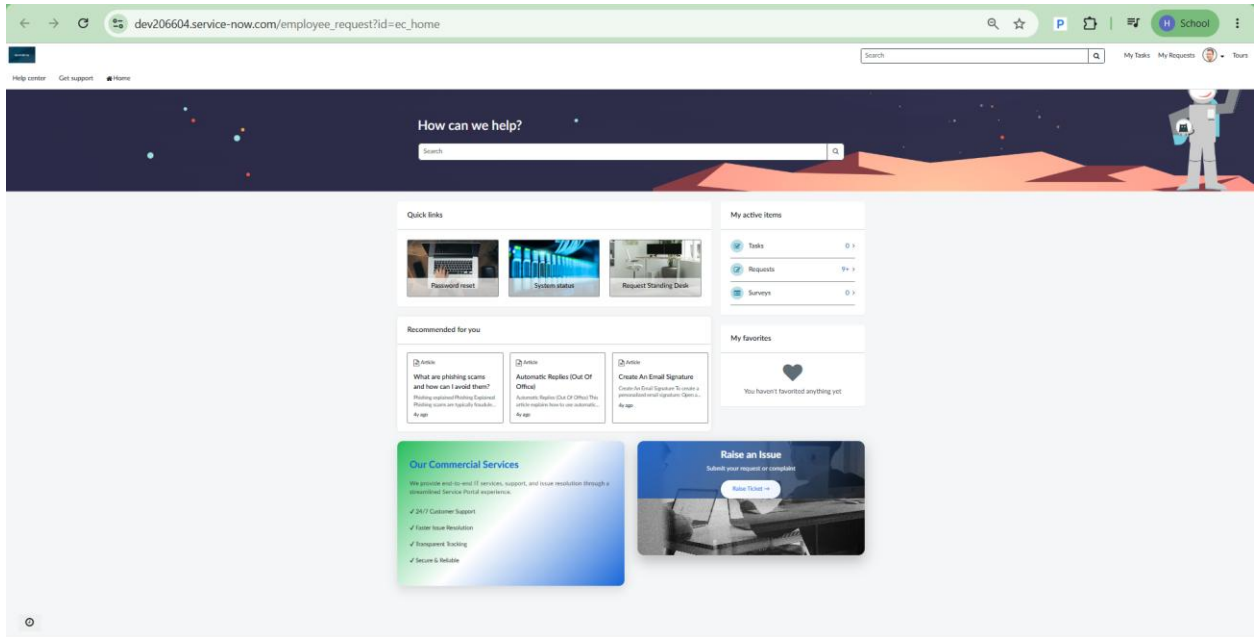


Figure 8.4: Final Employee Center Homepage with configured widgets

RESULT

- The two required widgets were successfully created and integrated into the existing Employee Center Homepage.
- The Commercial widget provides the required service information, while the Link redirect widget provides direct navigation to the Raise Employee Issue Record Producer.
- The 6×6 layout organizes both widgets into two equal columns, providing a simple and accessible employee-facing interface.

PHASE 7: TESTING AND VALIDATION

PHASE OVERVIEW

The seventh phase focuses on testing and validating the complete **Employee Raise Issue** solution. The functionality developed in the previous phases was tested through the Employee Center portal to ensure that the Record Producer, field dependency, widgets, and backend table work together correctly.

TESTING APPROACH

The solution was validated through an end-to-end workflow:

Employee Center → Raise Employee Issue → Enter Details → Select Category → Select Subcategory → Submit Issue → Verify Record

Test Scenarios

Test Case	Test Action	Expected Result	Status
TC01	Open Employee Center portal	Portal loads successfully	Passed
TC02	Access Raise Employee Issue	Issue submission form opens	Passed
TC03	Select Requester	Requester can be selected	Passed
TC04	Select Category	Available categories are displayed	Passed
TC05	Select Subcategory	Relevant subcategory is displayed based on Category	Passed
TC06	Enter issue details	Description can be entered successfully	Passed
TC07	Submit the issue	Issue is submitted successfully	Passed

TC08	Verify backend record	Corresponding record is created in Employee Raise issue table	Passed
-------------	-----------------------	---	--------

CATEGORY AND SUBCATEGORY VALIDATION

- The dependency between **Category** and **Subcategory** was specifically tested.
- For example, when **Network** is selected as the Category, the corresponding **VPN** subcategory is displayed. Likewise, the appropriate subcategory is displayed for the other configured categories.
- This confirms that the dependency configured in Phase 3 functions correctly during actual issue submission.

RECORD CREATION VALIDATION

- After submitting the issue through the Employee Center portal, the corresponding record was checked in the **Employee Raise issue** table.
- The submitted values were verified against the information entered in the Record Producer to ensure that the data was successfully transferred and stored.

END-TO-END VALIDATION

The complete workflow was successfully validated:

1. Employee accesses Employee Center

↓

2. Opens Raise Employee Issue

↓

3. Enters Requester and issue details

↓

4. Selects Category

↓

5. Relevant Subcategory is displayed



6. Employee submits the form



7. Record is created in Employee Raise issue table

RESULT

All major functional components of the Employee Raise Issue solution were tested successfully. The Employee Center portal, widgets, Record Producer, Category–Subcategory dependency, and backend record creation worked as expected.

The successful validation confirms that the implemented solution provides a functional end-to-end mechanism for employees to raise and track issues within ServiceNow.

PHASE 8 – CONCLUSION

CONCLUSION

The Employee Raise Issue – Record Producer & Service Portal Integration project was successfully implemented in ServiceNow. The solution provides a simple and structured way for employees to raise issues through the Employee Center portal. A custom scoped application, Employee Raise issue table, required fields, UI Policies, Category–Subcategory dependency, and Raise Employee Issue Record Producer were successfully configured. The Employee Center portal was also integrated with the required widgets and page layout. The complete workflow was tested to verify issue submission, field dependency, and backend record creation.

KEY OUTCOMES

- Created the Employee Center scoped application.
- Created the Employee Raise issue custom table and fields.
- Configured UI Policies and field dependency.
- Created the Raise Employee Issue Record Producer.
- Configured the Employee Center Service Portal.
- Added and configured the required widgets.
- Tested the complete issue submission process.
- Verified successful backend record creation.

FINAL RESULT

The completed solution provides a simple, organized, and user-friendly process for employees to raise issues through ServiceNow. The integration of the portal, Record Producer, custom table, fields, and widgets provides a complete end-to-end Employee Raise Issue solution.